

Explicacion delCodigo Agregado

Proyecto: ReStock-SaaS | Responsable: ArmandoValerio

PARTE 1: Pruebas Automatizadas (Jest, Supertest y k6)

Las pruebas estan estructuradas para validar el backend (logica de negocio y rutas), el frontend (componentes de la interfaz y validacion de formularios) y el rendimiento (pruebas de carga del servidor).

1. Configuracion de los Entornos de Pruebas

Backend Config (apps/api/jest.config.js)

- preset: 'ts-jest': Le dice a Jest que use el compilador ts-jest para transformar los archivos TypeScript (.ts) a JavaScript antes de correr los tests.
- testEnvironment: 'node': Configura el entorno de ejecucion como Node.js (ya que en el backend no necesitamos emular un navegador).
- moduleNameMapper: Permite que Jest entienda los alias de rutas como @/ mapeados al directorio raiz de desarrollo src/.
- collectCoverageFrom: Le indica a Jest que calcule la cobertura de codigo en todos los archivos .ts dentro de src, excluyendo el punto de entrada principal index.ts.

Frontend Config (apps/web/jest.config.js)

- nextJest: Importa la configuracion oficial de pruebas de Next.js. Esto carga automaticamente las variables de entorno de .env y habilita el compilador optimizado SWC.
- testEnvironment: 'jest-environment-jsdom': Configura un entorno de navegador simulado (JSDOM) en Node.js, lo cual es necesario para poder interactuar y renderizar componentes React en los tests.
- setupFilesAfterEnv: ['<rootDir>/jest-setup.js']: Carga el archivo de inicializacion de pruebas, el cual importa @testing-library/jest-dom para anadir funciones especiales como .toBeInTheDocument().

2. Pruebas del Backend (Jest + Supertest)

Validacion de Esquemas (apps/api/src/__tests__/schemas/ctaSchema.test.ts)

Comprueba que las reglas de validacion de datos hechas con Zod funcionen correctamente antes de procesar cualquier peticion HTTP:

- Caso Exitoso: Verifica que cuando se le pasa un objeto correcto (con nombre, email valido y origen), el validador de Zod de luz verde (result.success === true).
- Caso de Error de Email: Pasa un correo con formato invalido y valida que el sistema lo rechace y arroje el mensaje de error especifico para el campo email.
- Caso de Campos Faltantes: Intenta validar un objeto que no tiene correo electronico y comprueba que falle.

Pruebas de Servicios (apps/api/src/__tests__/services/ctaServices.test.ts)

Prueban la logica de negocio simulando la base de datos (con jest.mock) para no alterarla durante las pruebas:

- Caso Exitoso: Configura la base de datos falsa (mock) para retornar que no hay ningun correo duplicado. Comprueba que el servicio cree el registro de interes y retorne el objeto recién creado con su ID unico.
- Caso de Error Duplicado: Configura la base de datos para simular que el correo ya esta registrado y verifica que el servicio lance correctamente un error 'Email already registered', asegurando que no se intente guardar nada.

Pruebas de Rutas (apps/api/src/__tests__/routes/cta.test.ts)

Utiliza Supertest para levantar el servidor Express en memoria y enviar peticiones HTTP reales al endpoint /cta:

- POST /cta Exitoso (201): Valida que al enviar datos correctos, el API responda con el codigo HTTP 201 Created y los datos del nuevo registro.
- Errores de Validacion (400): Valida que si el correo no tiene formato valido o faltan campos obligatorios, el API responda con 400 Bad Request.

- Registro Duplicado (409): Verifica que si el servicio arroja un error de correo duplicado, la ruta lo capture y responda al usuario con un 409 Conflict.
- Fallo del Servidor (500): Simula que la base de datos se cayo y comprueba que la ruta responda de forma segura con un 500 Internal Server Error sin exponer detalles sensibles.

3. Pruebas del Frontend (Jest + React Testing Library)

Validaciones del Formulario (apps/web/___tests___/lib/validationsCTA.test.ts)

Prueba a nivel de codigo que el esquema de validacion en el frontend (interestSchema con Zod) sea estricto:

- Nombre: Acepta caracteres en espanol (como la N y acentos), pero rechaza nombres vacios, con numeros, menores a 2 caracteres o mayores a 100.
- Negocio: Valida reglas similares de longitud y caracteres especiales.
- Telefono: Verifica estrictamente que el numero comience con el prefijo internacional de Mexico (+52 o +52 seguido de exactamente 10 digitos). Si no cumple el formato, es rechazado.

Pruebas del Componente CTA (apps/web/___tests___/components/cta.test.tsx)

Simula el comportamiento de un usuario interactuando con el formulario de llamada a la accion:

- Renderizado: Asegura que el titulo, los campos (Nombre, Negocio, Telefono) y el boton de enviar aparezcan.
- Interaccion y Escritura: Simula al usuario escribiendo en los campos (userEvent.type) y valida que los inputs reflejen el texto introducido.
- Manejo de Errores Visuales: Simula que el usuario intenta enviar el formulario en blanco y verifica que aparezcan los errores visuales en color rojo (ej. 'al menos 2 caracteres', 'prefijo +52').
- Envio de Datos: Simula un envio correcto mockeando fetch para interceptar la llamada al backend (/cta), verifica que se manden los datos correctos y que los campos se limpien tras el exito.
- Persistencia (localStorage): Verifica que tras un registro exitoso, se actualice el contador en localStorage.

4. Pruebas de Rendimiento (k6)

Prueba de Disponibilidad (tests/k6/health-check.js)

Prueba rapida (smoke test) para validar que la raiz del API esta activa y respondiendo bajo carga basica:

- stages: Escala la carga de usuarios virtuales a 10 VUs en 10s, mantiene 10 VUs por 20s y baja a 0 en 10s.
- thresholds: Define limites de fallo si mas del 10% de peticiones fallan o el 95% de respuestas toman >500ms.

Prueba de Carga del Endpoint (tests/k6/cta-endpoints.js)

Simula un escenario de alta concurrencia real donde multiples usuarios se registran al mismo tiempo:

- Carga Incremental: Escala gradualmente de 0 a 50 usuarios concurrentes en un periodo de 45 segundos.
- Generacion Aleatoria: Usa datos dinamicos en cada iteracion para simular multiples usuarios reales y prevenir errores de base de datos por llaves unicas duplicadas.
- Metricas Personalizadas: Define las metricas cta_errors y cta_latency para trackear rendimiento y fallos.
- handleSummary: Genera un reporte personalizado y legible directamente en la consola mostrando porcentajes de exito, tiempos minimos, maximos, promedios y percentiles claves (p95 y p99).

PARTE 2: Apartado de "Inventario por Lotes"

Nota: Este codigo se encuentra en la rama remota 'feature/inventario-lotes'. A continuacion se detalla la logica e implementacion tecnica de esta solucion basandonos en sus criterios de aceptacion:

Definicion de Datos Ficticios (Mock Data)

Se crearon constantes estructuradas para representar LotInventario y Producto con formato de fecha YYYY-MM-DD y campos como cantidades, fecha de ingreso, caducidad, producto y estado, manteniendo la logica aislada en memoria.

Estados Derivados (Render Logic)

El componente calcula en tiempo real (en el renderizado) los estados clave:

- Lotes proximos a caducar: Calcula si quedan menos de 7 dias entre la fecha actual y la fecha de caducidad, marcandolos como PROXIMO_CADUCAR (destacado en amarillo/naranja con iconos).
- Stock Bajo: Evalua si la cantidad actual es inferior al limite minimo establecido y lo destaca en rojo.

Busqueda y Filtros en Memoria

Se aplican filtros con `.filter()` de JavaScript en base a los criterios de busqueda (por nombre/codigo) y selectores (por categoria, estado de caducidad/stock y rango de fechas), ejecutandose al instante sobre los mocks.

Interactividad del Dashboard

Se implemento una fila expandible y/o un modal para ver el detalle de cada lote. Los botones Editar, Eliminar y Agregar simulan la operacion manipulando el estado de React (`useState`), sin comunicarse con el backend y mostrando confirmaciones visuales.

Diseno Responsive

Diseno flexible adaptable que colapsa la tabla en tarjetas amigables en pantallas moviles y se muestra como tabla de datos completa en pantallas desktop.